ACTUAL

Dossier de projet

Application web de réservation de cours de sport

Information	Valeur
Candidat	Christophe Barrere
Certification	Titre professionnel Développeur web et web mobile
Référence	RNCP 37674 | Niveau 5
Technologies	Python, Django, HTML, CSS, Bootstrap et Bootswatch Simplex
Dépôt	https://github.com/ChristopheBarrere/actual_rncp
Version du dossier	Version 1 | Octobre 2026

Ce dossier présente la conception, le développement et la sécurisation de l'application Actual. Il s'appuie sur la version définitive documentée le 20 octobre 2026. Le passage à PostgreSQL et le déploiement public sont présentés comme les prochaines étapes du projet.

Sommaire

1 Synthèse du projet

2 Contexte et besoin

3 Objectifs et périmètre

4 Utilisateurs et parcours

5 Spécifications fonctionnelles

6 Choix techniques et environnement

7 Architecture de l'application

8 Conception de la base de données

9 Interfaces utilisateur et accessibilité

10 Développement back-end et règles métier

11 Sécurité

12 Tests et validation

13 Déploiement et stratégie de base de données

14 Organisation du projet et versionnement

15 Correspondance avec les compétences RNCP

16 Résultats, limites et améliorations

17 Conclusion

Annexes

Éléments restant à personnaliser

- captures d'écran finales après validation de l'interface ;
- URL de l'application déployée ;
- nom de l'organisme de formation et date de passage ;
- éventuels retours d'utilisateurs ou du formateur ;
- résultat du passage à PostgreSQL et du déploiement sur l'hébergement choisi.

1 Synthèse du projet

Actual est une application web responsive permettant à une salle de sport de publier ses cours et à ses adhérents de réserver une place en ligne. Le projet répond à un besoin simple : éviter une gestion manuelle des inscriptions tout en empêchant les doubles réservations et le dépassement de la capacité d'un cours.

J'ai développé l'application avec Python et Django en suivant l'architecture MVT. Le projet comprend une interface publique, un espace adhérent, une administration sécurisée, une base relationnelle SQLite, des contrôles de sécurité et des tests automatisés.

Domaine	État vérifié	Observation
Fonctionnalités utilisateur	Réalisées	Inscription, connexion, planning, détail, réservation, annulation et historique
Fonctions administrateur	Réalisées	CRUD des cours, réservations et utilisateurs dans l'administration Django
Base de données	Opérationnelle	SQLite pendant le développement local
Tests automatisés	À exécuter	4 tests couvrent la liste, le détail, la réservation et l'annulation
Contrôle Django	À exécuter	Commandes check et makemigrations --check documentées
Déploiement public	À réaliser	PostgreSQL et hébergement de production non encore configurés
Captures finales	À ajouter	À produire après validation visuelle définitive

2 Contexte et besoin

Une salle de sport propose plusieurs séances avec des horaires, des coachs et des capacités différentes. Lorsque les inscriptions sont gérées par téléphone, courrier électronique ou tableau papier, les adhérents ne connaissent pas toujours les places disponibles et le personnel doit contrôler les doublons manuellement.

L'application Actual centralise ces informations. Elle permet au visiteur de consulter le planning, à l'adhérent de gérer ses réservations et à l'administrateur de gérer les cours et les inscrits.

2.1 Problématique

Comment proposer un service de réservation simple et accessible tout en garantissant que le nombre d'inscrits ne dépasse jamais la capacité prévue et que seules les personnes autorisées accèdent aux fonctions de gestion ?

2.2 Réponse apportée

- un planning public des prochains cours ;
- un système de comptes utilisateurs basé sur l'authentification Django ;
- un historique personnel des réservations ;
- un espace de gestion réservé au personnel ;
- une base relationnelle assurant l'intégrité des données ;
- une interface responsive utilisable sur ordinateur et mobile.

3 Objectifs et périmètre

3.1 Objectifs principaux

- mettre en pratique les compétences front-end et back-end du RNCP 37674 ;
- construire une application complète à partir d'un besoin métier ;
- sécuriser les actions sensibles et contrôler les permissions ;
- modéliser une base relationnelle cohérente ;
- documenter l'installation, les tests et le déploiement ;
- rendre le projet reproductible à partir du dépôt GitHub.

3.2 Périmètre retenu

Inclus	Exclu volontairement
Comptes, authentification et autorisations	Paiement en ligne et gestion d'abonnements
Planning et détail des cours	Messagerie entre adhérents et coachs
Réservation, annulation et historique	Application mobile native
Gestion des cours et réservations dans l'administration	Statistiques commerciales avancées
SQLite en développement	Base NoSQL, non adaptée au besoin principal

4 Utilisateurs et parcours

Profil	Droits et besoins
Visiteur	Consulter l'accueil, le planning et le détail ; créer un compte ; se connecter
Adhérent	Réserver, annuler et consulter son historique personnel
Administrateur	Créer, modifier, supprimer ou annuler un cours ; consulter les inscrits ; gérer les comptes

4.1 Parcours principal d'un adhérent

1. Le visiteur ouvre le planning et sélectionne un cours.
2. S'il n'est pas connecté, l'application lui propose de se connecter ou de créer un compte.
3. L'utilisateur consulte la date, le coach, la durée et le nombre de places restantes.
4. Il confirme la réservation au moyen d'un formulaire POST protégé par CSRF.
5. La réservation apparaît dans son espace personnel.
6. Il peut ensuite l'annuler ; le statut passe alors à annulée.

4.2 Parcours administrateur

1. L'administrateur se connecte à l'administration Django avec un compte autorisé.
2. Il ouvre la liste des cours pour créer ou modifier un cours.
3. Les formulaires de l'administration valident les données côté serveur.
4. Il peut filtrer et rechercher les cours et les réservations.
5. Il peut également gérer les utilisateurs et leurs autorisations.

5 Spécifications fonctionnelles

5.1 Fonctionnalités utilisateur

Identifiant	Fonctionnalité	Critère d'acceptation
U01	Créer un compte	Le compte est créé lorsque UserCreationForm valide les informations
U02	Se connecter et se déconnecter	La session est ouverte ou fermée avec une redirection cohérente
U03	Consulter le planning	Les cours sont affichés avec leur disponibilité
U04	Consulter un cours	Date, coach, durée, capacité et places restantes sont visibles
U05	Réserver	Une place disponible produit une réservation confirmée
U06	Annuler	La réservation appartient à l'utilisateur et son statut devient annulée
U07	Consulter l'historique	L'utilisateur ne voit que ses propres réservations

5.2 Fonctionnalités administrateur

Identifiant	Fonctionnalité	Contrôle
A01	Créer un cours	L'administration Django contrôle l'accès
A02	Modifier ou supprimer	Les opérations sont réalisées depuis l'administration
A03	Annuler ou réactiver	Le champ is_cancelled contrôle la disponibilité
A04	Consulter les réservations	Recherche et filtres facilitent le suivi
A05	Gérer les utilisateurs	L'administration Django applique les permissions

5.3 Règles de gestion

1. Un cours possède au minimum une place.
2. Un utilisateur ne peut posséder qu'une réservation par cours.
3. Seule une réservation confirmée occupe une place.
4. Un cours annulé ou complet n'est plus réservable.
5. Une réservation annulée peut être réactivée si une place est encore disponible.
6. Les opérations de réservation et d'annulation exigent une requête POST.
7. Les fonctions de gestion sont protégées par l'administration Django.

6 Choix techniques et environnement

Technologie	Rôle	Justification
Python 3	Langage serveur	Syntaxe lisible et écosystème adapté au développement web
Django 5.2.6	Framework web	ORM, formulaires, authentification, administration et sécurité intégrés
HTML et templates Django	Structure des pages	Rendu serveur simple, accessible et maintenable

Technologie	Rôle	Justification
CSS, Bootstrap et Simplex	Présentation responsive	Composants cohérents et adaptation mobile
SQLite	Base locale	Installation immédiate et adaptée à l'apprentissage
PostgreSQL	Évolution prévue	Base envisagée pour la future mise en production
Git et GitHub	Versionnement	Historique, sauvegarde et partage du code

6.1 Environnement de développement

- Windows 11 avec WSL Ubuntu ;
- Visual Studio Code avec ouverture du projet dans WSL ;
- environnement virtuel Python .venv ;
- dépendances figées dans requirements.txt ;
- serveur de développement Django sur 127.0.0.1:8000.

7 Architecture de l'application

Django organise le projet selon le modèle MVT. Les URL dirigent la requête vers une vue. La vue contrôle les droits et les règles métier, interroge les modèles grâce à l'ORM, puis transmet les données à un template HTML.

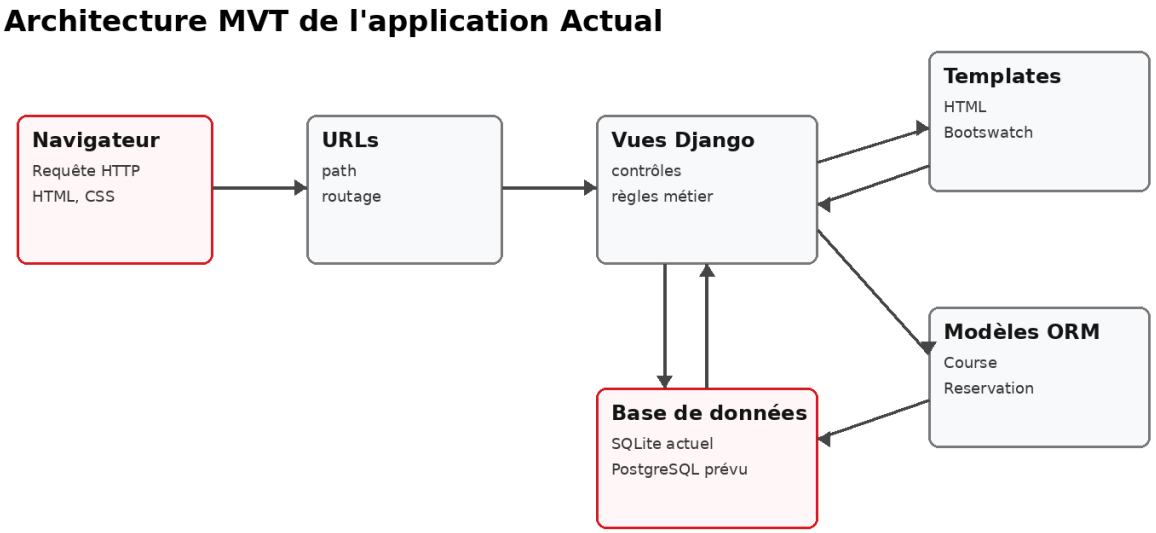


Figure 1 Architecture MVT appliquée au projet Actual

7.1 Répartition des responsabilités

Élément	Responsabilité dans Actual
actual	Configuration générale, routage principal, WSGI et ASGI
courses	Cours, réservations, inscription, vues, URL, administration et tests
templates	Template commun et pages propres aux fonctionnalités
static	Thème Simplex, feuille de style personnalisée, logo et image d'accueil

Élément	Responsabilité dans Actual
migrations	Historique des évolutions de la structure de la base

7.2 Arborescence principale

```
actual_rncp/
├── actual/
├── courses/
│   ├── migrations/
│   ├── static/courses/
│   ├── templates/
│   ├── admin.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── manage.py
├── requirements.txt
└── README.md
```

8 Conception de la base de données

La structure repose sur trois entités principales. Django fournit la table User. Le projet ajoute Course et Reservation. Reservation est la table d'association entre un utilisateur et un cours.

Modèle relationnel simplifié

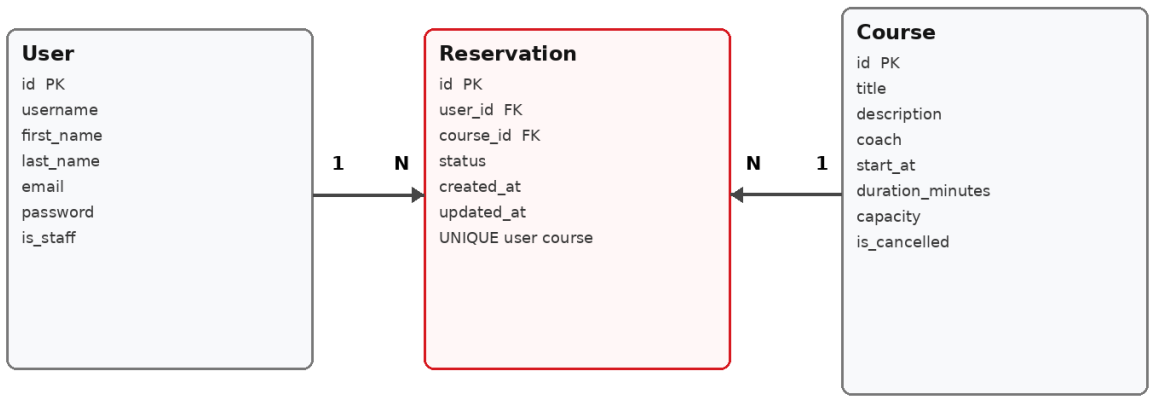


Figure 2 Modèle relationnel simplifié

8.1 Table Course

Champ	Type Django	Rôle
title	CharField 120	Titre du cours
description	TextField	Présentation de la séance
coach	CharField 120	Nom du coach
start_at	DateTimeField	Date et heure

Champ	Type Django	Rôle
duration_minutes	PositiveSmallIntegerField	Durée en minutes
capacity	PositiveSmallIntegerField	Nombre maximal de participants
is_cancelled	BooleanField	Cours annulé ou actif
created_at et updated_at	DateTimeField	Traçabilité de création et modification

8.2 Table Reservation

Champ	Type ou relation	Rôle
user	ForeignKey vers User	Adhérent concerné
course	ForeignKey vers Course	Cours réservé
status	confirmed ou cancelled	État de la réservation
created_at et updated_at	DateTimeField	Historique technique
contrainte	UniqueConstraint user course	Interdit une deuxième ligne pour le même couple

8.3 Calcul de la disponibilité

Le nombre de places restantes n'est pas stocké. Il est calculé à partir de la capacité du cours et du nombre de réservations confirmées. Cette solution évite de conserver deux valeurs susceptibles de devenir incohérentes.

```
places_available = max(
    capacity - confirmed_reservations_count,
    0,
)

is_full = places_available == 0
```

9 Interfaces utilisateur et accessibilité

L'interface utilise des templates Django communs et le thème Bootswatch Simplex. Le template base.html contient la navigation, les messages et les ressources partagées. Les autres pages héritent de cette structure.

9.1 Pages réalisées

Page	Objectif
Accueil	Présenter le service et orienter l'utilisateur vers le planning
Planning	Afficher les cours et leur disponibilité
Détail	Afficher les informations et l'action disponible
Inscription et connexion	Créer ou ouvrir un compte
Mes réservations	Afficher l'historique personnel et permettre l'annulation
Administration Django	Gérer les données et les utilisateurs

9.2 Responsive et accessibilité

- mise en page Bootstrap adaptée aux différentes largeurs d'écran ;
- navigation responsive avec bouton explicitement nommé ;
- libellés fournis par les formulaires Django ;

- structure de titres hiérarchisée ;
- formulaires utilisables au clavier ;
- états disponibles, complets et annulés indiqués par du texte en plus de la couleur.

9.3 Captures à intégrer dans la version finale

Capture	Preuve attendue
Accueil	Identité visuelle, navigation et appel vers le planning
Planning	Cartes, disponibilité et responsive
Détail d'un cours	Informations, places restantes et bouton de réservation
Mes réservations	Historique et annulation
Administration Django	Création ou modification d'un cours
Vue mobile	Adaptation de la navigation et des cartes

À compléter : *insérer des captures de la version finale, avec des données de démonstration ne contenant aucune donnée personnelle réelle.*

10 Développement back-end et règles métier

10.1 Inscription

La vue register utilise UserCreationForm, fourni par Django. Le formulaire applique les règles de validation des mots de passe, crée l'utilisateur puis le connecte automatiquement lorsque les données sont valides.

10.2 Consultation des cours

La vue course_list récupère les cours avec l'ORM Django et les transmet au template. Les propriétés du modèle calculent le nombre de réservations confirmées, les places disponibles et l'état complet du cours.

10.3 Réservation

La vue reserve_course exige une authentification et une requête POST. Elle refuse un cours annulé, un cours complet ou une réservation déjà confirmée. Une réservation annulée peut être réactivée ; sinon une nouvelle réservation est créée.

```
reservation = Reservation.objects.filter(
    user=request.user,
    course=course,
).first()

if course.is_cancelled or course.is_full:
    return redirect('course_detail', course_id=course.id)
```

10.4 Annulation

L'annulation ne supprime pas la réservation. Son statut devient cancelled. Ce choix conserve l'historique et autorise une réactivation ultérieure si le cours reste réservable.

10.5 Administration

Les modèles Course et Reservation sont enregistrés dans l'administration Django. Des colonnes, des filtres, une recherche et un ordre d'affichage facilitent la gestion. Les utilisateurs sont gérés avec le module d'authentification fourni par Django.

11 Sécurité

Risque	Mesure appliquée
Vol de mot de passe	Hachage et validateurs de mots de passe fournis par Django
Requête forgée	Jeton CSRF dans tous les formulaires POST
Injection SQL	Requêtes construites avec l'ORM Django
Accès sans connexion	Décorateur login_required
Accès administrateur abusif	Authentification et permissions de l'administration Django
Modification par URL GET	Réservation et annulation limitées à POST ; réponse 405 sinon
Double réservation	UniqueConstraint sur user et course
Dépassement de capacité	Contrôle de course.is_full avant la création
Code HTML injecté	Échappement automatique des variables dans les templates
Configuration de développement	SECRET_KEY et DEBUG à sécuriser avant la production

11.1 Jeton CSRF

Le jeton CSRF est une valeur secrète associée à la session et au formulaire. Django vérifie cette valeur lorsqu'une action POST est envoyée. Un site extérieur ne peut donc pas déclencher facilement une réservation ou une annulation à la place de l'utilisateur connecté.

```
<form method="post" action="{% url 'reserve_course' course.pk %}">
   {% csrf_token %}
   <button type="submit">Réserver ce cours</button>
</form>
```

11.2 Points à vérifier avant production

- définir une clé secrète longue et unique ;
- désactiver DEBUG ;
- renseigner précisément ALLOWED_HOSTS ;
- activer HTTPS avec un certificat valide ;
- changer immédiatement le mot de passe du compte de démonstration ;
- configurer puis sauvegarder régulièrement PostgreSQL ;
- ne pas charger de données personnelles réelles dans la démonstration.

12 Tests et validation

Le projet prévoit quatre tests automatisés écrits avec django.test.TestCase. Ils couvrent les principaux parcours et doivent être exécutés avant chaque livraison avec la commande python manage.py test.

Test	Résultat attendu	État
Afficher le planning	La réponse HTTP 200 contient le titre du cours	À exécuter

Test	Résultat attendu	État
Afficher le détail	La réponse HTTP 200 contient le nom du coach	À exécuter
Réserver connecté	Une réservation confirmée est créée	À exécuter
Annuler sa réservation	Le statut devient cancelled	À exécuter

12.1 Résultat à joindre

```
python manage.py test
python manage.py check
python manage.py makemigrations --check
```

Après exécution, une capture ou une copie du résultat OK devra être ajoutée au dossier final.

12.2 Tests manuels complémentaires

- navigation complète au clavier ;
- affichage sur smartphone, tablette et ordinateur ;
- messages de confirmation et d'erreur ;
- création, modification et suppression depuis l'administration Django ;
- installation complète depuis une nouvelle copie du dépôt ;
- déploiement PostgreSQL avec HTTPS.

13 Déploiement et stratégie de base de données

13.1 SQLite pendant le développement

SQLite est une base de données relationnelle. Elle satisfait donc le besoin pédagogique de modélisation et permet de lancer le projet sans installer de serveur de base de données. Elle reste adaptée au développement local et aux tests simples.

13.2 PostgreSQL pour la production

Pour une application publique, PostgreSQL est préférable. Il gère mieux les connexions simultanées, les comptes de base de données et les sauvegardes. Cette configuration n'est pas encore présente dans la version actuelle : elle constitue la prochaine étape technique.

Environnement	Base recommandée	État
Développement WSL	SQLite	Opérationnelle
Tests automatisés	SQLite temporaire	Opérationnelle
Production	PostgreSQL	À configurer puis à valider

13.3 Procédure de passage à PostgreSQL

1. Créer une base PostgreSQL et un utilisateur dédié.
2. Définir POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST et POSTGRES_PORT.
3. Lancer python manage.py migrate pour créer les tables.
4. Créer le superutilisateur de production.
5. Importer uniquement les données utiles si une reprise de données est nécessaire.
6. Exécuter les tests et python manage.py check --deploy.

13.4 Architecture de production prévue

- un hébergeur reçoit les requêtes HTTPS ;
- un serveur WSGI exécute l'application Django ;
- PostgreSQL conserve les données ;
- collectstatic rassemble les fichiers statiques ;
- les secrets sont fournis par l'environnement et non enregistrés dans Git.

13.5 État réel au moment du dossier

La version actuelle fonctionne localement avec SQLite et le serveur de développement Django. La configuration PostgreSQL, les variables d'environnement, la collecte des fichiers statiques, le certificat HTTPS et l'URL publique restent à réaliser. Le dossier devra être complété après cette étape.

14 Organisation du projet et versionnement

Le projet est organisé en étapes courtes : initialisation de Django, création des modèles, migrations, administration, vues, templates, authentification, réservations, interface, sécurité, tests et documentation. Chaque étape peut faire l'objet d'un commit Git simple et descriptif.

Phase	Exemple de résultat
Initialisation	Projet Django, environnement, dépendances et dépôt
Données	Modèles Course et Reservation, migrations et administration
Front-end	Templates, thème Simplex et responsive
Back-end	Authentification, réservations et administration
Qualité	Tests, sécurité et documentation pédagogique
Livraison	PostgreSQL, serveur WSGI et HTTPS à réaliser

14.1 Difficultés et solutions

Difficulté	Solution retenue
Calculer les places disponibles	Propriété calculée à partir des réservations confirmées
Empêcher un doublon de réservation	Contrainte d'unicité en base et contrôle dans la vue
Maintenir un thème cohérent	Template base commun et classes Bootstrap
Distinguer les droits	login_required pour l'adhérent et administration Django pour la gestion
Préparer la production	Conserver SQLite en local puis configurer PostgreSQL sur l'hébergement

15 Correspondance avec les compétences RNCP

Compétence du titre	Preuve dans Actual
Installer et configurer son environnement	WSL, environnement virtuel, requirements.txt, variables d'environnement
Maquetter des interfaces	Structure des écrans, parcours et captures à joindre
Réaliser des interfaces statiques	Templates HTML, Bootswatch Simplex, CSS responsive et composants Bootstrap

Compétence du titre	Preuve dans Actual
Développer la partie dynamique	Messages Django, formulaires et affichage conditionnel ; JavaScript personnel prévu en amélioration
Mettre en place une base relationnelle	User, Course, Reservation, clés étrangères, contrainte et migrations
Développer des composants d'accès aux données	ORM, all, filter, first, create, select_related et save
Développer des composants métier côté serveur	Inscription, disponibilité, réservation, annulation et administration
Documenter le déploiement	Compétence à finaliser avec PostgreSQL, variables, collectstatic et HTTPS

15.1 Choix concernant NoSQL

Le projet n'ajoute pas une base NoSQL artificiellement. Les utilisateurs, cours et réservations possèdent des relations et des contraintes fortes ; une base SQL convient donc mieux. Les principes NoSQL devront néanmoins être révisés pour l'entretien technique.

16 Résultats, limites et améliorations

16.1 Résultats obtenus

- parcours complet de l'inscription à l'annulation ;
- contrôle de la capacité et des doubles réservations ;
- séparation des droits adhérent et administrateur ;
- interface responsive cohérente avec le thème Simplex ;
- base relationnelle pilotée par l'ORM ;
- quatre tests automatisés couvrant le parcours principal ;
- documentation pédagogique permettant de reproduire le projet.

16.2 Limites actuelles

- application non encore publiée sur une URL de production confirmée ;
- PostgreSQL non encore configuré ;
- absence de JavaScript personnel dans la version actuelle ;
- tests d'interface et d'accessibilité encore principalement manuels ;
- absence de réinitialisation de mot de passe par courrier électronique configuré ;
- absence de pagination pour un volume important de cours ;
- pas de journal métier détaillé pour les opérations administratives ;
- pas de paiement, conformément au périmètre minimal choisi.

16.3 Améliorations prévues

- déployer l'application avec PostgreSQL et HTTPS ;
- ajouter des tests pour les cours complets, annulés, les doubles réservations et les permissions ;
- ajouter si possible une recherche ou un filtre en JavaScript ;
- automatiser les tests d'interface avec Playwright ou Selenium ;
- réaliser un audit Lighthouse et une vérification RGAA plus complète ;
- ajouter la récupération de mot de passe et la confirmation d'adresse électronique ;
- mettre en place une intégration continue GitHub Actions ;
- ajouter des sauvegardes automatisées de PostgreSQL.

17 Conclusion

Le projet Actual répond à son objectif principal : fournir une application web de réservation de cours, organisée autour d'une base relationnelle et de règles métier clairement identifiées. Il met en œuvre les composants essentiels d'un projet Django : modèles, vues, templates, formulaires, authentification, ORM, administration et tests.

La version actuelle constitue une base solide pour la présentation RNCP. Les dernières actions prioritaires consistent à configurer PostgreSQL, déployer l'application, exécuter et renforcer les tests, produire les captures finales et compléter les informations administratives.

Annexe A Routes principales

URL	Vue	Accès
/	home	Public
/cours/	course_list	Public
/cours/<id>/	course_detail	Public
/cours/inscription/	register	Public
/cours/connexion/	LoginView Django	Public
/cours/<id>/reserver/	reserve_course	Connecté et POST
/cours/<id>/annuler/	cancel_reservation	Connecté et POST
/cours/mes-reservations/	my_reservations	Connecté
/admin/	Administration Django	Personnel autorisé

Annexe B Installation locale

```
git clone https://github.com/ChristopheBarrere/actual_rncp.git
cd actual_rncp
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Annexe C Scénario de démonstration jury

1. Présenter rapidement le besoin et les trois profils utilisateurs.
2. Afficher l'accueil et le planning.
3. Ouvrir un cours et montrer les places disponibles.
4. Créer un compte ou se connecter avec un compte adhérent.
5. Réserver le cours et afficher Mes réservations.
6. Annuler la réservation et montrer le changement de statut.
7. Se connecter à l'administration et créer ou modifier un cours.
8. Afficher les réservations et la gestion des utilisateurs.
9. Présenter les modèles et la contrainte d'unicité.
10. Terminer par les tests et les prochaines étapes PostgreSQL et déploiement.

Annexe D Glossaire

Terme	Définition simple
Framework	Ensemble structuré d'outils et de règles facilitant le développement d'une application
ORM	Outil qui traduit les objets Python en requêtes vers une base relationnelle
MVT	Organisation Django en modèles, vues et templates

Terme	Définition simple
Migration	Fichier décrivant une évolution de la structure de la base
CSRF	Protection contre l'envoi d'un formulaire à l'insu de l'utilisateur
CRUD	Créer, lire, modifier et supprimer des données
Responsive	Interface qui s'adapte aux dimensions de l'écran
WSGI	Interface entre le serveur web et l'application Python

Annexe E Sources et documentation

- France Compétences, RNCP 37674, Développeur web et web mobile : https://www.francecompetences.fr/recherche/rncp/37674/
- Documentation Django 5.2 : https://docs.djangoproject.com/fr/5.2/
- Documentation Bootstrap 5 : https://getbootstrap.com/docs/5.3/
- Bootswatch Simplex : https://bootswatch.com/simplex/
- Documentation PostgreSQL : https://www.postgresql.org/docs/
- Dépôt du projet : https://github.com/ChristopheBarrere/actual_rncp

Annexe F Liste de contrôle avant remise

Vérification	État
Informations candidat et organisme complétées	À compléter
Captures finales insérées	À compléter
URL publique et PostgreSQL validés	À réaliser
Tests automatisés exécutés	À vérifier
Démonstration répétée	À planifier
Dossier relu et exporté en PDF	À faire après validation
Dépôt GitHub à jour	À vérifier avant remise
Aucun secret dans le dépôt	À vérifier avant remise